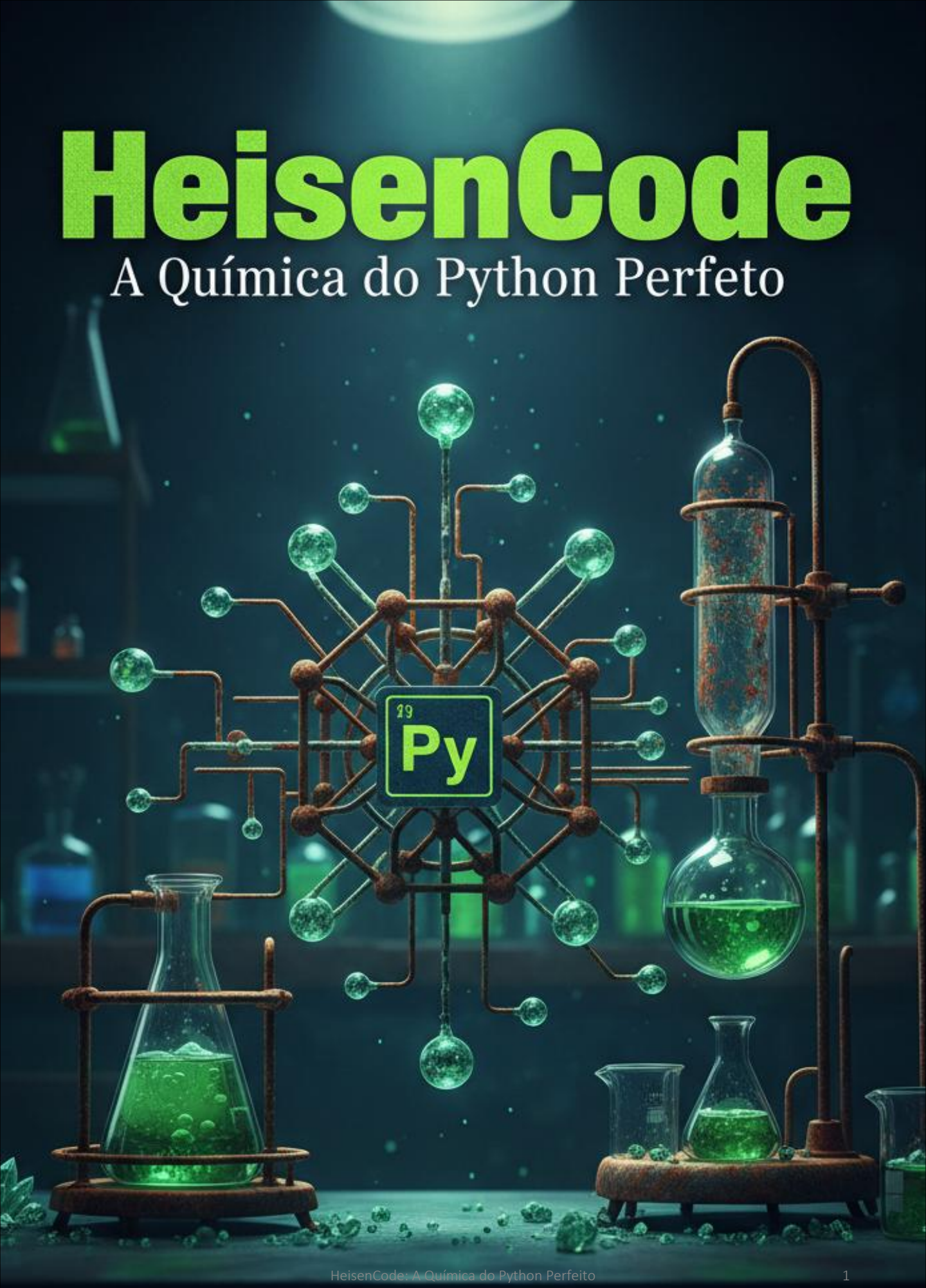
HeisenCode

A Química do Python Perfeto



01

Variáveis e Tipos: A Base de Tudo

Variáveis

Armazenam dados que seu programa vai usar.

Variáveis

```
nome = "Rafael"
idade = 30
saldo = 1500.75
ativo = True

print(f"{nome} tem {idade} anos e saldo de R${saldo}")
```

👉 Uso real: cadastro de usuário, dados de sistema, configs.

02

Condicionais: Tomando Decisões

Condicionais

Permitem que o código reaja a situações.

Condicionais

```
saldo = 500

if saldo ⩾ 1000:
    print("Cliente VIP")
elif saldo ⩾ 100:
    print("Cliente padrão")
else:
    print("Saldo baixo")
```

👉 Uso real: regras de negócio, validações, permissões.

03

Loops: Automatizando Repetições

Loops

Executam ações várias vezes.

Loops

```
for i in range(5):
    print(f"Processando item {i}")

#-------------------------------------

while True:
    comando = input("Digite 'sair' para encerrar: ")
    if comando == "sair":
        break
```

👉 Uso real: processamento de listas, automações, leitura contínua.

04

Listas: Trabalhando com Coleções

Listas

Armazenam múltiplos valores.

Listas

```
clientes = ["Ana", "João", "Carlos"]

for cliente in clientes:
    print(f"Enviando email para {cliente}")
```

👉 Uso real: listas de usuários, produtos, dados de API.

05

Dicionários: Dados Estruturados

Dicionários

Armazenam dados com chave e valor.

Dicionários

```
cliente = {
    "nome": "Rafael",
    "saldo": 1200
}

if cliente["saldo"] > 1000:
    print(f"{cliente['nome']} é VIP")
```

👉 Uso real: JSON, APIs, bancos de dados.

06

Funções: Reutilizando Código

Funções

Organizam e evitam repetição.

Funções

```
def calcular_desconto(valor, desconto):
    return valor * (1 - desconto)

preco_final = calcular_desconto(100, 0.1)
print(preco_final)
```

👉 Uso real: regras reutilizáveis, cálculos, lógica modular.

07

Tratamento de Erros: Evitando Falhas

Tratamento de erros

Previne que o sistema quebre.

Tratamentos

```
try:
    valor = int(input("Digite um número: "))
    print(valor * 2)
except ValueError:
    print("Entrada inválida!")
```

👉 Uso real: entrada de usuário, APIs instáveis, arquivos.

08

Manipulação de Arquivos

Manipulação de arquivos

Ler e escrever dados.

Arquivos

```
with open("dados.txt", "w") as arquivo:
    arquivo.write("Olá, mundo!")

with open("dados.txt", "r") as arquivo:
    print(arquivo.read())
```

👉 Uso real: logs, relatórios, armazenamento simples.

09

Consumindo APIs (Mundo Real)

Consumo de APIs

Python brilha aqui.

API

```
import requests

resposta = requests.get("https://api.github.com/users/octocat")
dados = resposta.json()

print(dados["login"], dados["public_repos"])
```

👉 Uso real: integração com sistemas, dados externos.

10

Automação Simples

Automação

Ganhe tempo com scripts.

Automação

```python
import os

arquivos = os.listdir()

for arquivo in arquivos:
    if arquivo.endswith(".txt"):
        print(f"Arquivo encontrado: {arquivo}")
```

👉 Uso real: organização de arquivos, scripts internos.

Agradecimentos

Agradecimentos

🧪 O Que Você Conquistou

Ao longo deste material, você aprendeu a:

Estruturar seu pensamento lógico
Escrever código limpo e funcional
Resolver problemas reais com Python
Dar os primeiros passos rumo a automação e sistemas

Isso já te coloca à frente de muita gente.

🧠 Mentalidade HeisenCode

Não busque perfeição no início. Busque progresso.

Código ruim hoje > código perfeito que nunca foi escrito
Pequenas evoluções > grandes ideias paradas
Consistência > motivação

🚀 Obrigado por Fazer Parte Disso

Obrigado por dedicar seu tempo a evoluir.

Esse é só o começo da sua jornada com Python.

E lembre-se:

👉 Quem domina o código, domina o jogo.

*** Feito por IA.**